INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM
Department of Computer Science and Engineering



Large-Scale Job Market Intelligence and Emerging Skill Demand Analysis

An End-to-End Big Data Architecture
Using HDFS, Apache Pig, and Apache Hive

Final Project Report

CSE412 — Big Data & Large-Scale Computing

Submitted by

N Medhashree	2023BCS0042
Ashmi S N	2023BCS0196
Deepanjali Tudu	2023BCS0066

Repository:
https://github.com/medhashree05/JOB_MARKET_ANALYSIS

Abstract

Online recruitment portals publish tens of thousands of unstructured job postings every week. Turning this data into labour-market intelligence needs a scalable pipeline that handles free text, multi-valued skill lists, inconsistent city names, and sparse salary information. This project builds an end-to-end batch pipeline over 97,929 Indian job postings using three layers of the Hadoop ecosystem: HDFS for storage and ingestion, Apache Pig for parallel normalisation, skill extraction and aggregation, and Apache Hive for structured HiveQL analytics over the Pig outputs. A seven-page Streamlit dashboard presents the results interactively.

The analysis covers role demand, skill demand, geographic hiring concentration, salary benchmarks and posting recency. Python, SAP, Java and SQL dominate skill demand, Bengaluru, Hyderabad and Pune hold 38.52% of all postings, and disclosed salaries are strongly right-skewed. A second pipeline implements the Skill Demand Velocity Index (SDVI) over dated monthly postings (January to June 2024) using Pig partitioning and Hive window functions, and feeds an interactive velocity dashboard.

Keywords: Big Data, HDFS, Apache Pig, Apache Hive, MapReduce, Job Market Analytics, Skill Demand, SDVI, Streamlit.

Contents

1 Introduction 1
1.1 Background and Motivation 1
1.2 Objectives 1
1.3 Scope 1

2 Literature Review 1

3 System Architecture and Environment 1
3.1 Architecture 1
3.2 Execution Environment 2
3.3 Repository Structure 2

4 Dataset and Stage 1: Ingestion and Cleaning 2
4.1 Dataset 2
4.2 Cleaning Pipeline 3

5 Stage 2: Transformation with Apache Pig 3

6 Stage 3: Analytics with Apache Hive 4

7 Results 5
7.1 Role Demand 5
7.2 Skill Demand 5
7.3 Geographic Hiring Concentration 6
7.4 Salary Analytics 6
7.5 Posting Recency 7

8 Presentation Layer: Streamlit Dashboard 7

9 Skill Demand Velocity Index (SDVI) Pipeline 8
9.1 Motivation and Definition 8
9.2 Pipeline 8
9.3 Quadrant Classification 9
9.4 Results for June 2024 9

10 Reproducing the Project 9

11 Limitations 10

12 Conclusion and Future Work 10

1 Introduction

1.1 Background and Motivation

Labour-market intelligence informs curriculum planning, workforce development and hiring strategy. Recruitment data, however, is noisy: titles and cities are written inconsistently, skills are packed into one comma-separated field, salaries are often undisclosed or in diflerent currencies, and posting dates are frequently relative ("3 Days Ago") . Processing such data at scale calls for a distributed, fault-tolerant architecture rather than single-node tools.

1.2 Objectives

Ingest, clean and validate a large raw job-posting dataset into HDFS.

Transform and normalise it with Apache Pig, including skill extraction and grouped aggregation.

Build Hive tables and answer analytical questions on roles, skills, locations, salary and recency.

Present the results through an interactive dashboard.

Implement a velocity-based skill-demand metric (SDVI) over dated monthly data.

1.3 Scope

The primary pipeline (Sections 3-8) runs on the Indian Job Market Dataset 2025. Because that dataset carries only relative recency text, month-by-month skill trends are produced by the separate SDVI pipeline (Section 9) on dated postings.

2 Literature Review

Dean and Ghemawat [1] introduced MapReduce for parallel processing on commodity clusters with automatic partitioning and fault recovery. HDFS [2] provides the underlying block-replicated storage with a central NameNode and distributed DataNodes. Olston et al. [3] proposed Pig Latin, a dataflow language whose operators (`FILTER`, `FOREACH`, `GROUP`, `FLATTEN`) compile to MapReduce jobs, while Thusoo et al. [4] introduced Hive, which layers SQL-like querying and schema-on-read tables over files in HDFS. White [5] describes how these components are combined in practice.

For labour-market analytics, Khaouja et al. [6] survey skill-extraction methods and note that rule-based tokenisation suits high-throughput batch pipelines, and Djumalieva and Sleeman [7] show that taxonomies built from vacancy text can reveal emerging skills early. Ranking skills by raw frequency favours long-established tools, while plain percentage growth over-rewards tiny counts (1 to 3 postings is +200%). Velocity metrics that weight growth by volume, such as the SDVI used here, address both problems.

3 System Architecture and Environment

3.1 Architecture

The pipeline is a staged, decoupled batch architecture (Figure 1). Each stage reads the previous stage's output from HDFS, so stages can be re-run independently.

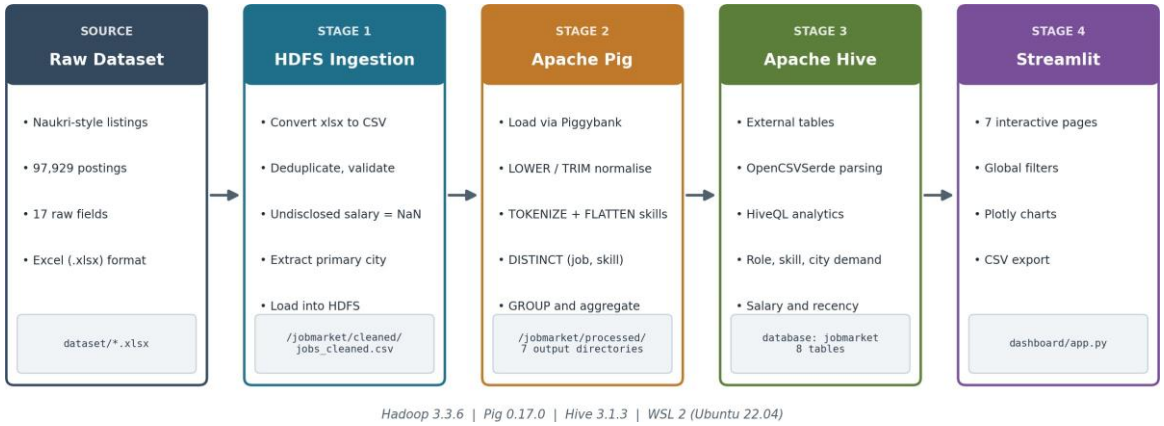


Figure 1: End-to-end architecture: raw dataset, HDFS ingestion, Apache Pig, Apache Hive and Streamlit dashboard.

3.2 Execution Environment

Component	Version	Path / configuration
Operating system	Ubuntu 22.04 LTS (WSL 2)	Windows Subsystem for Linux
Java	OpenJDK 1.8	`JusrJlibJjvmJjava-8-openjdk-amd64`
Distributed file system	Apache Hadoop 3.3.6	`JusrJlocalJhadoop`, replication factor 1
Transformation	Apache Pig 0.17.0	`JusrJlocalJpig`, Piggybank enabled
Data warehouse	Apache Hive 3.1.3	`JusrJlocalJhive`, database `jobmarket`
Dashboard	Python 3.13, Streamlit 1.39, Plotly	local virtual environment

Table 1: Software stack.

3.3 Repository Structure

```
dataset/              source Excel dataset
task1_hdfs/           ingestion, cleaning, HDFS load (scripts 01-04)
task2_pig/            Pig transfornations (task2_jobnarket.pig, runner)
task3_hive/           Hive DDL and analysis queries (scripts 01-04)
dashboard/            Streanlit app (app, data_loader, analytics, charts)
ashni_sdvi_pipeline/ SDVI pipeline: Pig, Hive vindov queries, dashboard
docs/                 report, presentation, screenshots
```

4 Dataset and Stage 1: Ingestion and Cleaning

4.1 Dataset

The Indian Job Market Dataset 2025 contains 97,929 Naukri-style postings with 17 raw fields. Table 2 describes the cleaned schema that Stage 1 produces and Pig consumes.

Column	Type	Description
job_id	string	Unique posting identifier
job_title	string	Posting title
company_name, company_id	string	Employer name (“Unknown” if missing) and id
location	string	Cleaned primary city, used for grouping
location_rav	string	Original multi-location text
currency	string	INR or USD
min_Jmax_Javg_salary	float	Annual salary; empty when not disclosed
experience_rav	string	e.g. “2-4 Yrs”
min_Jmax_experience	float	Years of experience
skills	string	Comma-separated skills (“Not Specified” if missing)
posted_rav	string	Relative recency text, e.g. “6 Days Ago”
revievs_count, rating	float	Company reviews and rating (often missing)

Table 2: Cleaned schema (jobs_cleaned. csv).

4.2 Cleaning Pipeline

Stage 1 is implemented by four scripts: 01_convert_xlsx_to_csv. py, 02_clean_data. py, 03_hdfs_load. sh and 04_verify_hdfs. sh. The cleaning decisions are summarised in Table 3.

Issue in raw data	Decision
Duplicate rows and job_ids (250)	Exact duplicates removed (247); a further 3 removed by job_id; first occurrence kept.
Missing company_name (4)	Filled with “Unknown”.
Missing skills (571)	Filled with “Not Specified”.
Salary “Not disclosed” (64,076)	Raw min/max of 0 converted to NaN so unknown is not mistaken for zero. “Unpaid” postings (333) keep 0.
Multi-location strings	Primary city extracted (e.g. “Kolkata(Chinar Park)” becomes “Kolkata”); original kept in location_rav.
Relative recency text	Kept as posted_rav; no absolute dates are invented.
Inconsistent salary/experience (min > max)	Rows removed.

Table 3: Stage 1 cleaning decisions.

The result is 97,929 raw rows → 97,679 cleaned rows (99.74% retained), loaded to /jobmarket/cleaned/jobs_cleaned. csv (raw copy at /jobmarket/raw/jobs_raw. csv).

5 Stage 2: Transformation with Apache Pig

The script task2_jobmarket. pig loads the cleaned CSV with Piggybank's CSVExcelStorage, removes the header, applies an explicit schema and performs the following steps.

1. Normalise job titles and locations with LOWER(TRIh()).
2. Explode skills: FLATTEN(TOKENIZE(skills, ’,’)), trim, drop empty and “not specified” tokens, then DISTINCT on (job_id, skill) so a posting counts once per skill.
3. Aggregate posting counts by skill, by location and by role.

4. Salary statistics: job count, minimum, maximum and mean salary by role and by location, grouped by currency so INR and USD are never mixed.
5. Recency binning: `posted_raw` mapped to 0-1 Days, 1-7 Days, 7-30 Days, 30+ Days and Unknown.

Output (under `JjobmarketJprocessedJ`)	Content	Used by Hive table
`jobs`	Normalised posting records	`jobs`
`skills`	Distinct (job, skill) pairs	`job_skills`
`skills_demand`	Postings per skill	`skills_demand`
`location`	Postings per primary city	`job_locations`
`salary`	Salary statistics by role and currency	`job_salary`
`salary_location`	Salary statistics by city and currency	`salary_by_location`
`time_based`	Postings per recency bucket	`time_based`

Table 4: Pig output directories and their Hive tables.

The runner `run_task2_pig. sh` configures Java, Hadoop and Pig, starts the MapReduce JobHistoryServer, runs Pig with `-stop_on_failure` and checks that every output directory exists before Hive starts. Output is written as quoted CSV so commas inside titles and company names are preserved.

6 Stage 3: Analytics with Apache Hive

Hive maps external tables over the Pig outputs in the `j obmarket` database using `OpenCSVSerde`, which preserves quoted fields containing commas. Data stays in HDFS; Hive adds only table metadata.

Listing 1: External table over the Pig skill output.

```
CREATE EXTERNAL TABLE IF NOT EXISTS job_skills (
  job_id STRIN6, job_title STRIN6, location STRIN6,
  currency STRIN6, skill STRIN6
)
ROW FORHAT SERDE 'org. apache. hadoop. hive. serde2. OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/jobnarket/processed/skills';
```

Analysis	HiveQL logic	Purpose
Top job roles	`GROUP BY job_title`, count, `LIhIT 10`	Most advertised roles
Skill demand	`COUNT(DISTINCT job_id)` per skill	Postings requesting each skill
Location demand	`job_locations` ordered by count	Hiring hubs
Salary by role	`avg_salary > 0`, `HAVING COUNT(*) >= 5`	Benchmarks with enough data
Skill–role pairs	`GROUP BY job_title, skill`	Skills most tied to a role
Posting recency	`time_based` ordered by count	Freshness of postings

Table 5: Hive analysis queries (`03_analysis. sql`).

Listing 2: Skill-demand query.

```
SELECT LOWER(TRIH(skill)) AS skill, COUNT(DISTINCT job_id) AS job_count
FROH job_skills
WHERE skill IS NOT NULL AND TRIH(skill) <> '' AND job_id IS NOT NULL
6ROUP BY LOWER(TRIH(skill))
ORDER BY job_count DESC LIHIT 20;
```

The eight tables are `jobs`, `job_skills`, `skills_demand`, `job_locations`, `job_salary`, `salary_by_location`, `time_based` (all external) and the managed table `skill_trends`.

7 Results

7.1 Role Demand

Rank	Job role	Postings	Share (%)
1	application developer	2,033	2.08
2	application lead	1,346	1.38
3	sales executive	583	0.60
4	software development engineer	497	0.51
5	business development executive	457	0.47
6	trust & safety new associate	348	0.36
7	data engineer	339	0.35
8	business development manager	325	0.33
9	sales manager	322	0.33
10	java full stack developer	285	0.29

Table 6: Top 10 job roles ($N = 97{,}679$).

Application developer and application lead together account for 3,379 postings (3.46%), reflecting IT-services demand.

7.2 Skill Demand

Rank	Skill	Unique postings	Penetration (%)
1	sales	9,178	9.40
2	python	5,846	5.99
3	project management	5,543	5.67
4	customer service	5,122	5.24
5	sap	5,043	5.16
6	management	4,849	4.96
7	css	4,186	4.29
8	java	4,088	4.19
9	sql	3,983	4.08
10	business development	3,738	3.83

Table 7: Top 10 skills (each posting counted once per skill).

Python is the leading programming language (5,846 postings), followed by Java (4,088) and SQL (3,983). SAP (5,043) shows strong enterprise ERP demand.

7.3 Geographic Hiring Concentration

Rank	Primary location	Postings	National share (%)
1	bengaluru	18,320	18.76
2	hyderabad	10,899	11.16
3	pune	8,405	8.60
4	mumbai	6,478	6.63
5	chennai	6,099	6.24
6	gurugram	5,002	5.12
7	noida	4,627	4.74
8	ahmedabad	2,377	2.43
9	remote	1,952	2.00
10	kolkata	1,794	1.84

Table 8: Top 10 hiring hubs.

Bengaluru leads with 18,320 postings (18.76%), about 1.7 times Hyderabad. Bengaluru, Hyderabad and Pune together hold 37,624 postings (38.52%). Fully remote postings are only 2.00%, so on-site and hybrid work dominate.

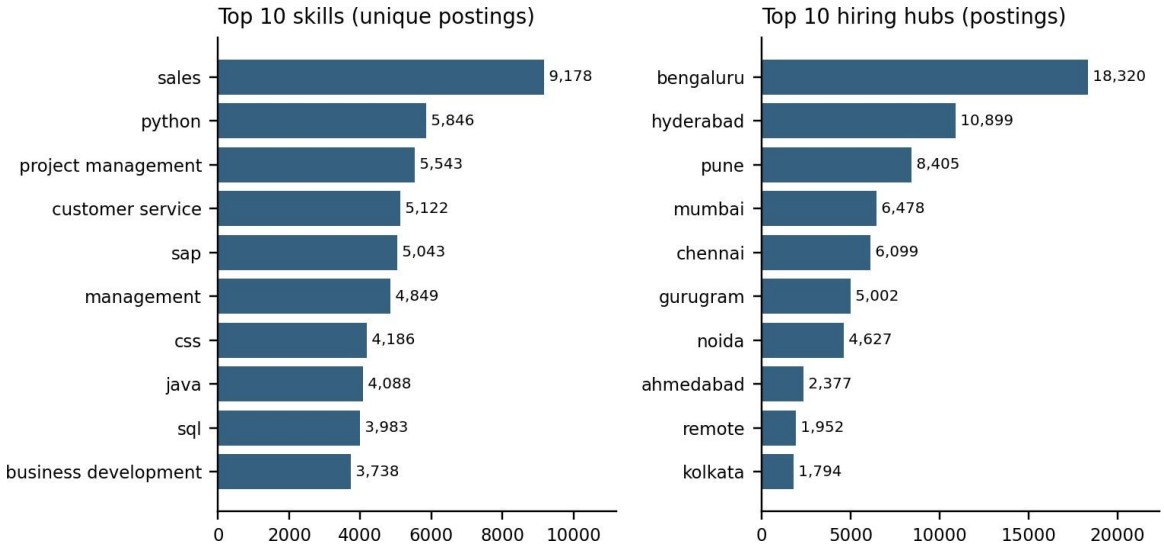


Figure 2: Top 10 skills and top 10 hiring hubs.

7.4 Salary Analytics

Of 97,550 INR postings, 33,124 (33.96%) disclose salary; of 129 USD postings, 20 do. Currencies are analysed separately. Disclosed INR salaries have a mean of INR 757,776 against a median of INR 425,000, a clear right skew driven by technology and executive roles.

Job role	Jobs	Mean (INR)	Min (INR)	Max (INR)
deputy manager*	9	9,749,778	85,000	82,500,000
medical oncologist consultant	5	7,850,000	4,500,000	10,750,000
radiologist consultant	6	5,291,667	4,500,000	5,500,000
lead data engineer	5	3,380,000	1,100,000	5,750,000
servicenow architect	5	2,850,000	2,250,000	3,250,000
data architect	12	2,775,000	1,125,000	6,300,000
data solution architect†	37	2,500,000	2,500,000	2,500,000
solution architect	59	2,471,441	115,000	3,750,000
senior software engineer	22	2,228,864	840,000	3,750,000

Table 9: Salary by role (INR, roles with at least 5 disclosed postings). *One posting of INR 82,500,000 lifts the mean; the role's median is INR 700,000. †All 37 postings advertise a fixed INR 2,500,000.

Outside specialised healthcare consultancies, the highest technology pay is in data engineering and architecture (lead data engineer INR 3.38M, data architect INR 2.78M, solution architect INR 2.47M).

7.5 Posting Recency

Recency category	Postings	Share (%)
1–7 Days	86,140	88.19
Unknown	8,248	8.44
0–1 Days	2,753	2.82
30+ Days	538	0.55

Table 10: Posting recency distribution.

Over 88% of postings were up to a week old when collected, so the dataset is a current snapshot of the market.

8 Presentation Layer: Streamlit Dashboard

The dashboard in `dashboard/` is a presentation layer only; it does not re-implement the pipeline. It has four modules:

`data_loader.py`: loads Hive/Pig exports from `dashboard/data/` if present, otherwise applies the Stage 1 cleaning rules to the source dataset; the active source is always shown on screen.

`analytics.py`: pure calculation functions (KPIs, distributions, experience buckets).

`charts.py`: Plotly figure builders.

`app.py`: page routing, sidebar filters and session state.

Pages: (1) Executive Overview with KPI cards and automated insights; (2) Skill Demand Intelligence with top-20 skills, drill-downs and co-occurrence; (3) Salary Analytics by role, location, skill and experience; (4) Location and Hiring; (5) Experience and Job Roles (Entry 0-2, Mid 2-5, Senior 5+ years); (6) Emerging Skill Trends; (7) Data Explorer with search, column filters and CSV download.

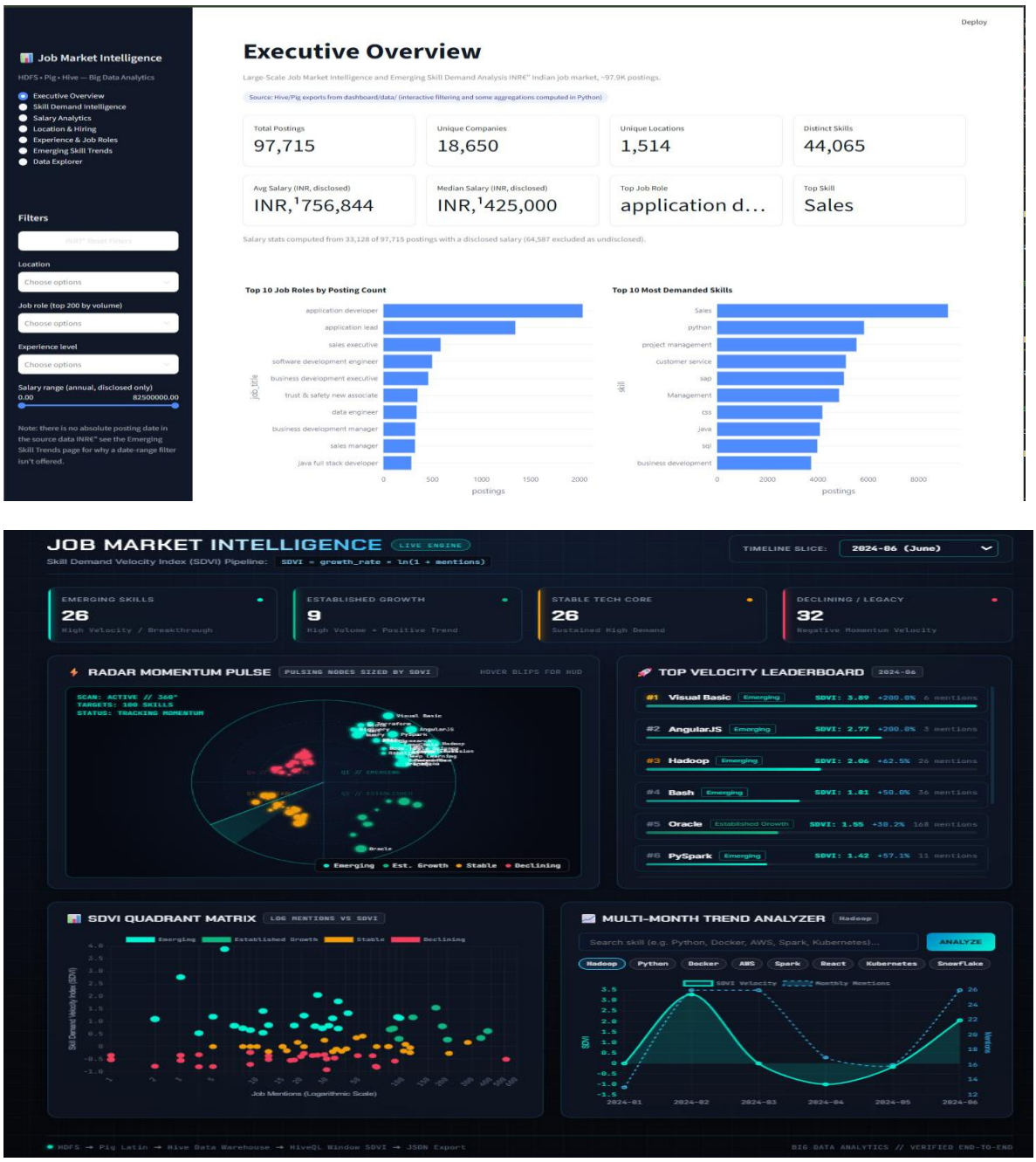


Figure 3: Dashboard: Executive Overview and Skill Demand Intelligence .

9 Skill Demand Velocity Index (SDVI) Pipeline

9.1 Motivation and Definition

Raw frequency keeps long-established skills on top, and plain percentage growth exaggerates tiny counts. The SDVI scales monthly growth by the logarithm of volume:

$$\text{growth_rate} = \frac{\text{Mentions}_t - \text{Mentions}_{t-1}}{\text{Mentions}_{t-1}}, \tag{1}$$

$$\text{SDVI} = \text{growth_rate} \times \ln(1 + \text{Mentions}_t). \tag{2}$$

For a low-volume jump ($2 \to 4$) SDVI ≈ 1.61, while a high-volume rise ($100 \to 150$) gives SDVI ≈ 2.51. Stable incumbents have SDVI near 0 and declining tools have negative SDVI.

9.2 Pipeline

The pipeline (Figure 4) processes 10,000 technology postings dated January to June 2024 against a 100-skill taxonomy, and lives in `ashmi_sdvi_pipeline/`.

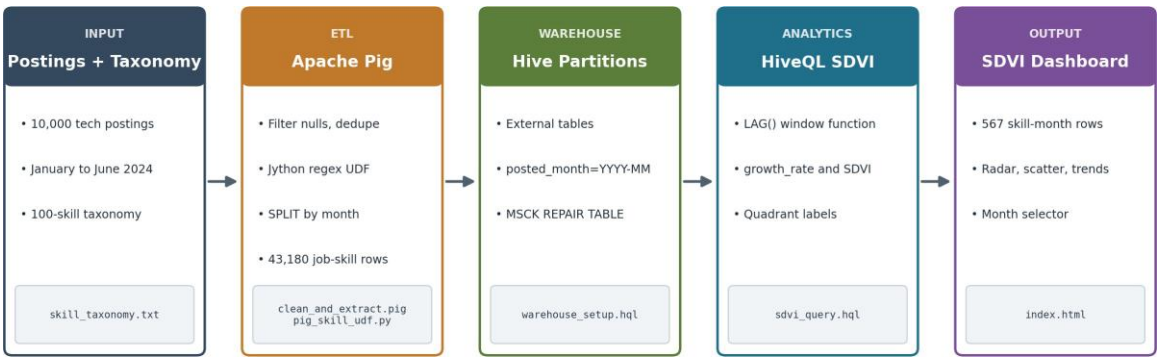


Figure 4: SDVI pipeline from dated postings to the velocity dashboard.

Pig ETL (`clean_and_extract.pig`): filters null rows, deduplicates, normalises location, derives `posted_month`, extracts skills with a Jython UDF (`pig_skill_udf.py`) using case-insensitive word-boundary regexes (so "Java" does not match "JavaScript"), and uses `SPLIT` to write `posted_month=YYYY-hh` directories. This produces 43,180 job-skill rows.

Hive warehouse (warehouse_setup. hql): external tables j obs and j ob_skills partitioned by posted_month, registered with hSCK REPAIR TABLE.

HiveQL SDVI (sdvi_query. hql): monthly mentions per skill, LAG() over each skill for the previous month, then growth rate, SDVI and quadrant. Window functions are used because Pig has no native LAG.

Export and dashboard: results are exported to JSON (567 skill-month records) and shown in index. html.

Listing 3: Core of the SDVI window query.

```
LA6(nentions, 1) OVER (PARTITION BY skill ORDER BY nonth) AS prev_nentions
...
CASE WHEN prev_nentions IS NULL OR prev_nentions = 0 THEN 0.0
     ELSE ROUND((nentions - prev_nentions) / prev_nentions, 4) END AS grovth_rate
...
ROUND(grovth_rate * LN(1.0 + nentions), 4) AS sdvi
```

9.3 Quadrant Classification

Each (skill, month) record is labelled by the rules in Table 11, evaluated from top to bottom.

Order	Condition	Label
1	growth_rate < -0.08	Declining
2	growth_rate ≥ 0.20 and mentions ≤ 100	Emerging
3	mentions ≥ 75 and growth_rate ≥ 0.04	Established Growth
4	$\lvert$growth_rate$\rvert \leq 0.18$	Stable
5	growth_rate > 0	Emerging
6	otherwise	Declining

Table 11: Quadrant rules used in sdvi_query. hql.

9.4 Results for June 2024

Quadrant	Skills	Avg SDVI	Examples
Emerging	26	+1.68	Visual Basic, AngularJS, Hadoop, Bash, Ant
Established Growth	9	+0.84	Oracle, DevOps, Machine Learning
Stable	26	+0.12	Python, SQL, Linux, Git, Docker, Spark
Declining	32	−0.68	SVN, Snowflake, Kubernetes, Terraform

Table 12: SDVI quadrants for June 2024.

Hadoop rose to 26 mentions (+62.5%, SDVI +2.06) and Oracle to 168 mentions (+30.2%, SDVI +1.55), while Python and SQL stayed high and flat across all six months. The dashboard (index. html) oflers a month selector, quadrant KPI cards, a top-velocity leaderboard, a log-mentions versus SDVI scatter, and a per-skill six-month trend analyser.

10 Reproducing the Project

Listing 4: Run order from the repository root (WSL).

```
# Stage 1: ingestion and cleaning
python3  task1_hdfs/scripts/01_convert_xlsx_to_csv. py
python3 task1_hdfs/scripts/02_clean_data. py
start-dfs. sh ãã start-yarn. sh ãã hdfs dfsadnin -safenode leave
bash task1_hdfs/scripts/03_hdfs_load. sh
bash  task1_hdfs/scripts/04_verify_hdfs. sh

# Stage 2: Pig,  then Stage 3: Hive
bash task2_pig/scripts/run_task2_pig. sh
bash  task3_hive/scripts/run_task3_hive. sh

# Stage 4: dashboard
streanlit run dashboard/app. py

# SDVI pipeline
bash  ashni_sdvi_pipeline/run_full_pipeline. sh
```

11 Limitations

The primary dataset has relative recency text only, so skill trends over time come from the separate dated SDVI dataset.

Normalisation is limited to case and whitespace; synonyms (e.g. “sde” and “software development engineer”) are not merged.

Only about 34% of postings disclose salary; undisclosed values are excluded rather than treated as zero.

The pipeline ran on a single WSL 2 node, and the dashboard works from exported files rather than live cluster services.

12 Conclusion and Future Work

This project delivers a working HDFS-Pig-Hive pipeline with a Streamlit dashboard over 97,679 verified Indian job postings, plus an SDVI pipeline that measures monthly skill momentum with Hive window functions. Enterprise software roles and foundational skills (Python, SAP, Java, SQL) dominate demand, 38.52% of postings are in Bengaluru, Hyderabad and Pune, and data engineering and architecture lead technology pay.

Future work: scheduled multi-month scraping into dated HDFS directories; a synonym and occupation ontology; applying SDVI to the main dataset once dated; a minimum-mention threshold for SDVI; and workflow orchestration with Apache Airflow.

References

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in Proc. OSDI, 2004, pp. 137-150.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in Proc. IEEE MSST, 2010, pp. 1-10.

[3] C. Olston, B. Reed, U. Srivastava, R. Kumar, and A. Tomkins, "Pig Latin: A not-so-foreign language for data processing," in Proc. AOM SIGMOD, 2008, pp. 1099-1110.

[4] A. Thusoo et al., "Hive: A warehousing solution over a map-reduce framework," Proc. VŁDB Endowmení, vol. 2, no. 2, pp. 1626-1629, 2009.

[5] T. White, Hadoop: The Definiíive Guide, 4th ed. O'Reilly Media, 2015.

[6] S. Khaouja, I. Kassou, and M. Ghogho, "A survey on skill identification from online job ads," IEEE Access, vol. 9, pp. 118434-118454, 2021.

[7] J. Djumalieva and C. Sleeman, "An open and data-driven taxonomy of skills extracted from online job adverts," Nesta Working Paper No. 18/03, 2018.